

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering



A report on RUET Hall Management System

Course Code: CSE 3100

Course Title: Web Based Application Project

Authors

ANIRUDDHA ROY

2203081

B

MD ASHFAQ AHAMED

2203161

C

TAREQ ABRAR

2203166

C

MD SOYEB AZAM SEFAT

2203172

C

Supervisor

UTSHA DAS

Assistant Professor

Dept. of CSE, RUET

Date: 24/06/2026

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology
Department of Computer Science & Engineering



Certification

This is to certify that the project titled: **RUET Hall Management System** submitted by (*Aniruddha Roy*, 2203081),(*MD ASHFAQ AHAMED*,2203161),(*TAREQ ABRAR*,2203166), (*MD SOYEB AZAM SEFAT*, 2203172) for the fulfillment of the requirements of the course CSE 3100 (Web Based Application Project), has been examined and approved by the undersigned.

The project has been carried out under my supervision, and it is recommended for submission and evaluation.

Supervisor:

Name: Utsha Das

Designation: Assistant Professor

Department: Dept. of Computer Science & Engineering

Institution: Rajshahi University of Engineering & Technology

Date: 24-06-2026

Signature: _____

Contents

Chapter 1	7
Introduction	7
1.1 Background of the Project	7
1.2 Objectives	7
1.3 Scope of the Project	8
1.4 Report Organization	8
Chapter 2	9
Background Study	9
& Related Work	9
2.1 Theoretical Foundation	9
2.2 Overview of Existing Systems	10
2.3 Comparative Analysis	10
2.4 Identified Gaps	11
2.5 Justification of Proposed System	11
Chapter 3	12
System Analysis	12
& Requirements	12
3.1 Stakeholder Analysis	13
3.2 Functional Requirements	13
3.3 Non-Functional Requirements	15
3.4 Feasibility Study (Engineering & Society)	15

Chapter 4	16
System Design	16
4.1 Architecture Design	16
4.2 Database Design	17
4.3 User Interface Design	23
Chapter 5	27
Implementation	27
5.1 Development Environment	27
5.2 Technologies Used (Modern Tools)	27
5.3 Module-wise Implementation	27
5.4 Code Structure & Key Snippets	27
Chapter 6	27
Testing & Validation	27
6.1 Testing Strategy	28
6.2 System Testing	28
6.3 User Acceptance Testing	28
Chapter 7	30
Project Management	30
& Teamwork	30
7.1 Project Planning	30
7.2 Individual & Team Work	30
7.3 Timeline	31
7.4 Budget & Cost Analysis	31
7.5 Risk Management	32
Chapter 8	32
Results & Discussion	32

8.1 System Output	33
8.2 Performance Evaluation	36
8.3 Comparison with Existing Systems	37
Chapter 9	37
Conclusion &	37
Future Work	37
9.1 Conclusion	38
9.2 Limitations	38
9.3 Future Enhancements	38

Abstract

The traditional hall management system in residential halls at Rajshahi University of Engineering & Technology (RUET) heavily relies on manual paper-based processes for seat allocation, dining meal token distribution, utility fee collection, and complaint management. These manual operations are highly time-consuming, prone to calculation errors, and impose a significant administrative burden on hall authorities while causing long queues and dissatisfaction among students. To eliminate these challenges, a web-based Hall Management System has been developed.

The proposed system digitizes the entire lifecycle of a residential student. Through this website, students can sign up, create profiles, and apply for hall seats online. The Hall Provost can review, evaluate, and approve these applications seamlessly via a dedicated admin dashboard. Furthermore, the system introduces a digital meal token management module, allowing students to purchase dining tokens online. This automates the financial tracking for Hall Managers, providing them with instantaneous, error-free calculations of daily tokens sold and total revenue.

Additionally, a built-in damage reporting and complaint module enables students to report broken furniture or facility issues directly to the authorities with descriptions.

Ultimately, this system simplifies complex administrative workflows, reduces paperwork, ensures financial transparency, and enhances the overall quality of hospitality service within RUET halls. Developed using modern web technologies, this scalable application serves as a foundational step toward building a smart, paperless campus.

List of Figures

Figure 1 ER Diagram	23
Figure 2 Student Dashboard	23
Figure 3 Hall Administrator Dashboard	24
Figure 4 Meal Token Management Interface	25
Figure 5 Damage Reporting Interface	26
Figure 6 Login Page	33
Figure 7 Student Dashboard	34
Figure 8 Hall Application Page	34
Figure 9 Token Book	35
Figure 10 Complaint Management Page	36
Figure 11 Admin Dashboard	36

List of Tables

Table 1 Comparison of Existing and Proposed Systems	10
Table 2 Sample Test Cases	28
Table 3 Project Timeline	31
Table 4 Estimated Cost Analysis	31
Table 5 Risk Assessment	32

Table 6 Performance Evaluation	36
Table 7 Comparison with Existing Systems	37

Chapter 1

Introduction

1.1 Background of the Project

Residential halls are an essential part of student life at Rajshahi University of Engineering & Technology (RUET). Currently, many hall-related activities such as seat allocation, fee management, meal token distribution, and complaint handling are performed manually. These traditional processes require significant paperwork and human effort, making them inefficient and prone to errors.

Students often face difficulties in applying for hall seats, purchasing meal tokens, and reporting maintenance issues. Similarly, hall managers and provosts must spend considerable time maintaining records and managing student information manually.

With the rapid advancement of information technology, web-based management systems have become an effective solution for automating administrative tasks. Therefore, this project proposes a web-based Hall Management System that digitizes hall operations and provides a centralized platform for students and hall authorities.

1.2 Objectives

To develop a web-based Hall Management System that automates and simplifies hall administration processes at RUET.

Specific Objectives:

- To provide online hall seat application facilities for students.
- To automate hall seat approval and management processes.
- To enable digital meal token purchasing and tracking.
- To make online payments .
- To maintain accurate records of student payments and dues.
- To assist hall managers in monitoring meal token distribution and financial information.
- To provide a complaint and maintenance reporting system for students.
- To reduce paperwork and administrative workload.
- To improve transparency, efficiency, and communication within residential halls.

1.3 Scope of the Project

The proposed system covers the major functionalities required for efficient hall management.

Included Features

- Student registration and login.
- Hall seat application and approval.
- Student information management.
- Digital meal token purchase system.
- Complaint and maintenance request submission.
- Administrative dashboard for hall managers and provosts.
- Report generation and monitoring facilities.

Limitations

- The system is currently designed specifically for RUET residential halls.
- High server load may affect performance if a very large number of users access the system simultaneously.
- The current reporting system focuses mainly on hall management activities and may require additional customization for future administrative requirements.
- Multi-language support has not been implemented in the current version.

1.4 Report Organization

This report is organized into several chapters.

- **Chapter 1** introduces the project, objectives, scope, and report structure.
- **Chapter 2** presents background studies, related works, and justification of the proposed system.
- **Chapter 3** discusses system analysis, stakeholder requirements, and feasibility studies.
- **Chapter 4** describes the system architecture, database design, and user interface design.
- **Chapter 5** explains implementation details, development tools, and technologies used.
- **Chapter 6** presents testing strategies, system validation, and user acceptance testing.
- **Chapter 7** discusses project planning, teamwork, timeline, budget, and risk management.
- **Chapter 8** presents results, outputs, and performance evaluation.
- **Chapter 9** concludes the project, highlights limitations, and suggests future enhancements.

Chapter 2

Background Study & Related Work

2.1 Theoretical Foundation

With the advancement of information technology, web-based management systems have become an essential part of modern organizations and institutions. Universities are also adopting digital solutions to automate administrative activities and improve operational efficiency.

Residential hall management is one of the important administrative sectors in a university. In many cases, hall-related activities such as seat allocation, meal token distribution, fee collection, and complaint management are performed manually. These manual processes require significant effort, consume time, and increase the possibility of human errors.

To overcome these challenges, a web-based Hall Management System has been developed. The system provides a centralized platform where students can apply for hall seats, purchase meal tokens, check payment information, and submit complaints. At the same time, hall authorities can manage student records, monitor transactions, and generate reports efficiently.

The system utilizes database management concepts to store and retrieve information securely. User authentication and authorization mechanisms are also implemented to ensure that only authorized users can access specific functionalities.

2.2 Overview of Existing Systems

Currently, many residential halls in universities operate through manual procedures. Students submit paper applications for seats, meal token distribution is handled manually, and payment records are maintained in physical registers.

Several universities have introduced partial digital solutions, but many systems only support seat allocation or payment management separately. Most systems lack an integrated platform that combines hall applications, meal management, payment tracking, and complaint handling.

2.3 Comparative Analysis

Table 1 Comparison of Existing and Proposed Systems

Features	Manual System	Existing Systems	Partial Proposed System
Online Hall Application	No	Yes	Yes

Features	Manual System	Existing Systems	Partial Proposed System
Seat Approval Management	Manual	Partial	Yes
Meal Token Management	No	No	Yes
Payment Tracking	Manual	Partial	Yes
Complaint Submission	No	Limited	Yes
Centralized Database	No	Yes	Yes
Automated Reports	No	Limited	Yes

2.4 Identified Gaps

After studying existing approaches, the following limitations were identified:

- Heavy dependence on manual paperwork.
- Time-consuming seat allocation procedures.
- Difficulty in maintaining payment records.
- Lack of digital meal token management.
- Limited communication between students and hall authorities.
- No centralized complaint management system.
- High possibility of human errors in record keeping.

2.5 Justification of Proposed System

The proposed Hall Management System addresses the identified limitations by providing a centralized web-based platform.

The system automates hall applications, seat approval, payment management, and meal token distribution. Students can access services conveniently while

administrators can monitor operations efficiently. The complaint management feature further improves communication and maintenance response. As a result, the system reduces administrative workload, increases transparency, minimizes errors, and improves overall hall management efficiency.

Chapter 3

System Analysis & Requirements

3.1 Stakeholder Analysis

The proposed RUET Hall Management System is designed to serve multiple stakeholders involved in residential hall operations. Each stakeholder interacts with the system according to their responsibilities and requirements.

Students

Students are the primary users of the system. They can register, log in, apply for hall seats, purchase meal tokens, make payments, view dues, receive notifications, and submit damage reports regarding hall facilities.

Hall Administrators

Hall administrators manage day-to-day hall operations. They review seat applications, allocate rooms, verify payments, manage meal menus, monitor token sales, approve expenses, and generate reports.

Hall Provost

The provost oversees overall hall management activities. The provost can monitor occupancy information, review reports, track financial records, and supervise administrative activities.

System Administrator

The system administrator is responsible for maintaining the system, ensuring database security, managing user roles, and resolving technical issues.

3.2 Functional Requirements

The system provides several integrated modules to support hall management operations.

Student Management Module

This module allows students to:

- Register and create accounts.
- Log in securely.
- Update profile information.
- View hall and room information.
- Receive system notifications.

Seat Management Module

This module handles:

- Online seat applications.
- Application review and approval.
- Seat allocation.

- Room occupancy tracking.
- Session-wise hall management.

Meal Management Module

This module allows:

- Creation of daily meal menus.
- Meal item management.
- Meal token booking.
- Lunch and dinner selection.
- Monitoring available meal quantities.

Payment Management Module

The payment system supports:

- Hall fee payment.
- Meal payment.
- Payment verification.
- Receipt management.
- Due calculation and tracking.

Damage Reporting Module

Students can:

- Report damaged hall property.
- Upload supporting evidence.
- Track complaint status.

Administrators can:

- Verify reports.
- Estimate repair costs.
- Mark issues as resolved.

Notification Module

The notification system provides:

- Announcement delivery.
- Student notifications.
- Administrative notices.
- Read/unread status tracking.

Report Generation Module

The system generates:

- Seat allocation reports.
- Meal sales reports.
- Payment summaries.
- Expense reports.

- Damage report summaries.
- PDF reports for hall authorities.

3.3 Non-Functional Requirements

Security

The system must ensure secure authentication and authorization. Passwords are stored in encrypted form, and role-based access control restricts unauthorized access.

Reliability

The system should provide accurate information and remain available during normal operation without data loss.

Performance

Database queries and report generation should be completed within an acceptable response time.

Usability

The interface should be simple, responsive, and easy to use for both students and administrators.

Maintainability

The system architecture should support future modifications and feature additions.

Scalability

The application should support additional halls, rooms, students, and administrators without major structural changes.

Availability

The system should be accessible through modern web browsers on desktop and mobile devices.

3.4 Feasibility Study (Engineering & Society)

Technical Feasibility

The system is technically feasible because it is developed using modern web technologies, relational database design principles, and secure authentication mechanisms. The architecture supports all required functionalities including payment processing, meal token management, and report generation.

Economic Feasibility

The project requires minimal development cost due to the use of open-source technologies. The automation of administrative activities reduces paperwork and operational costs, making the system economically beneficial.

Social Feasibility

The system improves communication between students and hall authorities while increasing transparency in hall operations. It reduces manual workload and provides convenient access to services.

Ethical Considerations

The system maintains confidentiality of student information and ensures that personal data is accessed only by authorized users.

Accessibility

The web application is designed to be responsive and can be accessed using computers, laptops, tablets, and smartphones.

Chapter 4

System Design

4.1 Architecture Design

The RUET Hall Management System follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Database Layer. This architecture ensures modularity, scalability, and efficient data management.

Presentation Layer

The Presentation Layer provides the user interface through which students and hall administrators interact with the system. It includes web pages for registration, login, hall seat application, meal token booking, payment management, damage reporting, and administrative monitoring.

Application Layer

The Application Layer contains the business logic of the system. It processes user requests, validates data, manages authentication, handles payment operations, generates reports, and controls communication between the user interface and the database.

Database Layer

The Database Layer stores all information related to students, halls, rooms, seat applications, seat allocations, meal menus, meal tokens, payments, notifications, expenses, and damage reports. A relational database management system is used to ensure data consistency and integrity.

4.2 Database Design

A relational database has been designed to efficiently manage hall operations. The database consists of multiple interconnected tables that support student management, seat allocation, meal management, payment processing, and complaint handling.

The database is designed following normalization principles to minimize redundancy and maintain consistency.

Major Entities

The primary entities of the system are:

- Uni_Students
- Hall_Admins
- Halls
- Rooms
- Academic_Sessions
- Seat_Applications
- Seat_Allocations
- Meal_Menus
- Meal_Menu_Items
- Meal_Tokens
- Meal_Payments
- Payments
- Student_Dues
- Damage_Reports
- Expenses
- Notifications
- Notification_Reads

Student Entity

The **Uni_Students** table stores student information including personal details, academic information, hall assignments, and account status.

Key Attributes:

- Student ID
- Email
- Name
- Phone Number
- Roll Number
- Academic Department
- Academic Session
- Hall
- Room ID
- Verification Status

Hall Entity

The **Halls** table maintains information about residential halls.

Key Attributes:

- Hall Name
- Address
- Contact Number
- Total Capacity
- Total Rooms
- Active Status

Room Entity

The **Rooms** table stores room-related information.

Key Attributes:

- Room Number
- Hall
- Capacity
- Current Occupancy
- Room Status

Seat Application Entity

The **Seat_Applications** table stores online hall seat applications submitted by students.

Key Attributes:

- Student ID
- Roll Number
- Hall

- Academic Session
- Application Status
- Review Information

Seat Allocation Entity

The **Seat_Allocations** table records approved seat assignments.

Key Attributes:

- Student ID
- Room ID
- Hall
- Allocation Date
- Allocated By

Meal Management Entities

The meal management system consists of:

Meal_Menus

Stores daily meal schedules and menu details.

Meal_Menu_Items

Stores individual food items included in a menu.

Meal_Tokens

Stores meal token booking information.

Key Attributes:

- Student ID
- Menu ID
- Meal Date
- Meal Type
- Quantity
- Total Amount
- Booking Time

Payment Management Entities

The payment subsystem consists of:

Payments

Stores hall fee payment records.

Meal_Payments

Stores meal-related payment transactions.

Student_Dues

Tracks unpaid dues and payment status.

Key Attributes:

- Due Amount
- Due Status
- Paid Date
- Hall Information

Damage Report Entity

The **Damage_Reports** table allows students to report damaged hall property.

Key Attributes:

- Student ID
- Hall
- Location Description
- Asset Details
- Image URL
- Description
- Estimated Cost
- Verification Status
- Repair Information

Expense Entity

The **Expenses** table records hall expenditures.

Key Attributes:

- Hall
- Expense Title
- Amount
- Category
- Approved By

Notification Entity

The notification subsystem consists of:

Notifications

Stores announcements and system notifications.

Notification_Reads

Tracks whether a user has read a notification.

Database Relationships

The following relationships exist within the database:

- One Hall can contain many Rooms.
- One Hall can accommodate many Students.
- One Student can submit multiple Seat Applications.
- One Student can receive one Seat Allocation at a time.
- One Student can purchase multiple Meal Tokens.
- One Meal Menu can contain multiple Meal Items.
- One Student can make multiple Payments.
- One Student can submit multiple Damage Reports.
- One Hall Administrator can manage multiple operations and reports.
- One Notification can be read by multiple users.



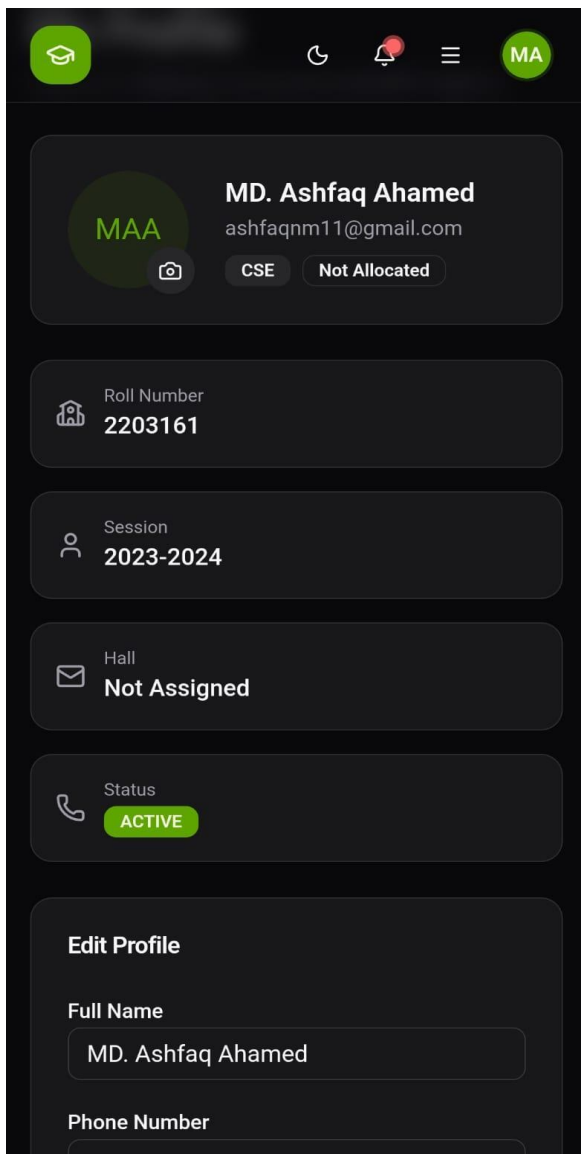
4.3 User Interface Design

The user interface has been designed with a focus on simplicity, responsiveness, and usability.

Student Dashboard

The student dashboard provides quick access to:

- Profile Information
- Hall Seat Application
- Meal Token Booking
- Payment History
- Due Information
- Damage Reporting
- Notifications



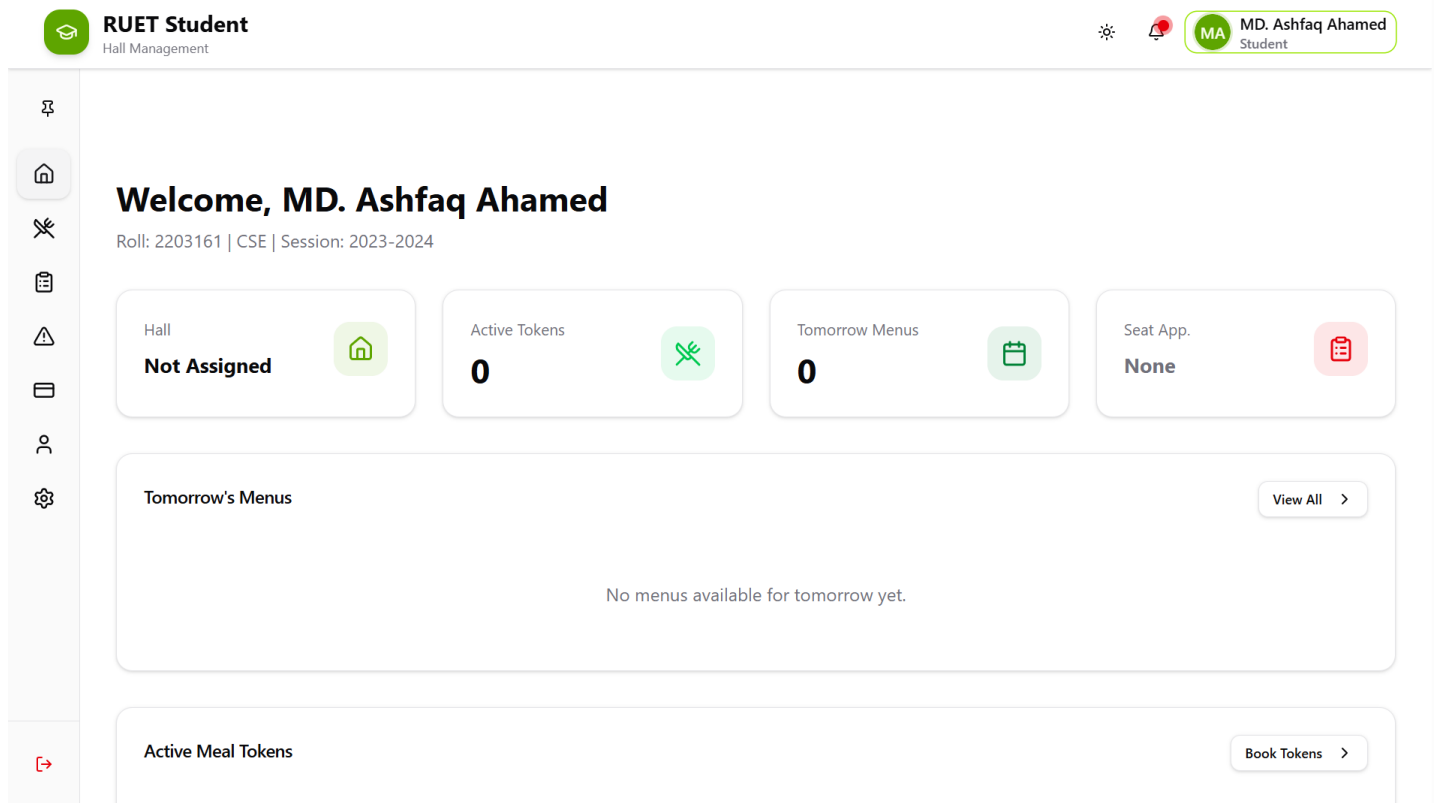
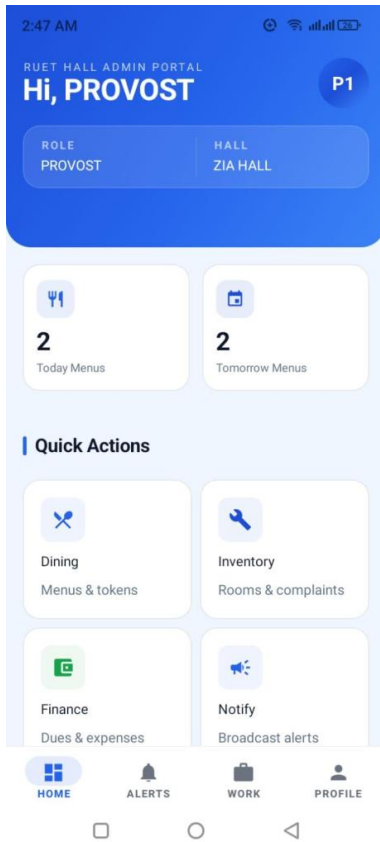


Figure 2 Student Dashboard

Hall Administrator Dashboard

The hall administrator dashboard provides:

- Seat Application Review
- Room Allocation Management
- Meal Monitoring
- Payment Verification
- Expense Tracking
- Damage Report Management
- PDF Report Generation



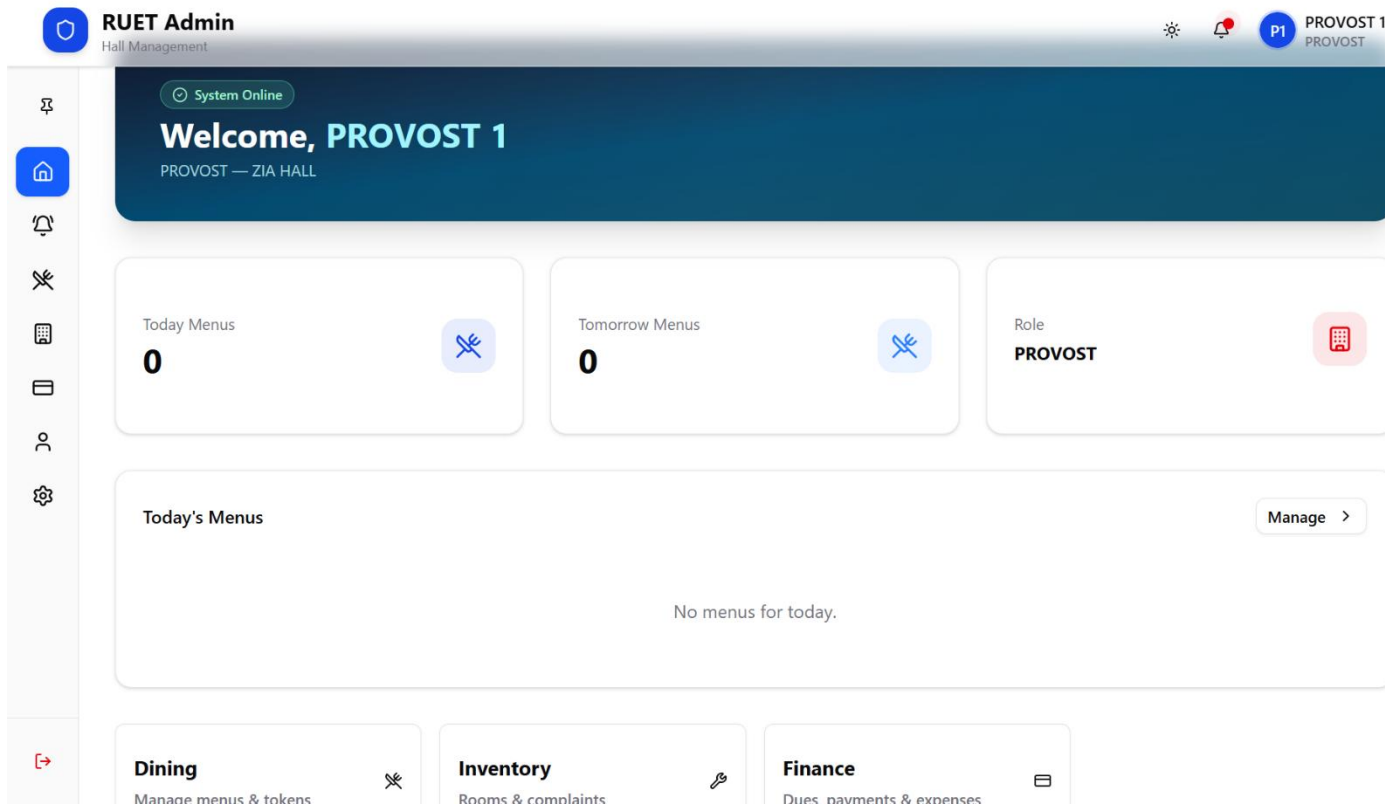


Figure 3 Hall Administrator Dashboard

Payment Interface

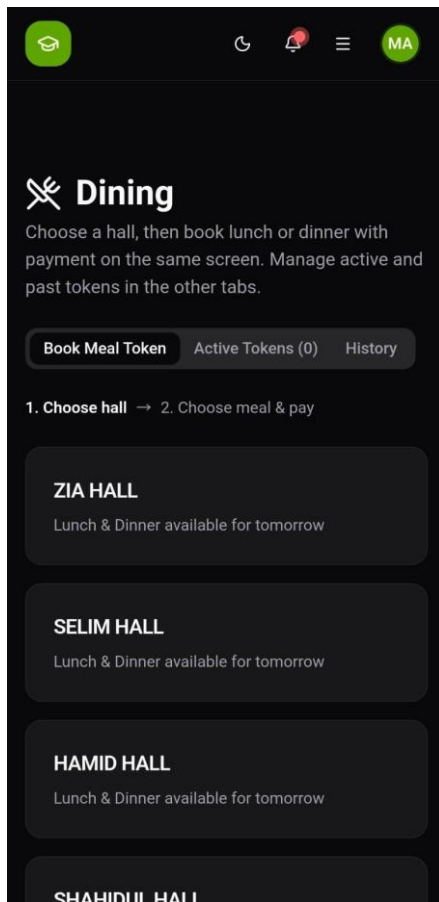
The payment interface enables students to complete online transactions and view payment records.

Figure 4.5: Online Payment Interface

The image shows a mobile application interface for a demo payment system. At the top, there is a blue circular logo with a white cursor icon and the word "DEMO". To the right of the logo are icons for a document with a checkmark and a close button (X). Below the logo, the word "Demo" is displayed. Underneath, there are four circular icons: a headset for "Support", a question mark for "FAQ", a gift box with a "3" for "Offers", and a door for "Login". A blue navigation bar contains three tabs: "CARDS", "MOBILE BANKING", and "NET BANKING". The "CARDS" tab is selected. Below the tabs, there are logos for "VISA", "MasterCard", and "Other Cards". A text input field is labeled "Enter Card Number". Below this are two input fields: "MM/YY" and "CVC/CVV", with a small card icon next to the CVC/CVV field. Below these is a "Card Holder Name" input field. At the bottom of the form, there is a checkbox labeled "Save card & remember me" and a question mark icon. Below the checkbox, it says "By checking this box you agree to the [Terms of Service](#)". At the very bottom, there is a large grey button with a hand icon and the text "PAY 50 BDT".

Meal Token Interface

Students can select meal types, view menus, and book meal tokens through this interface.



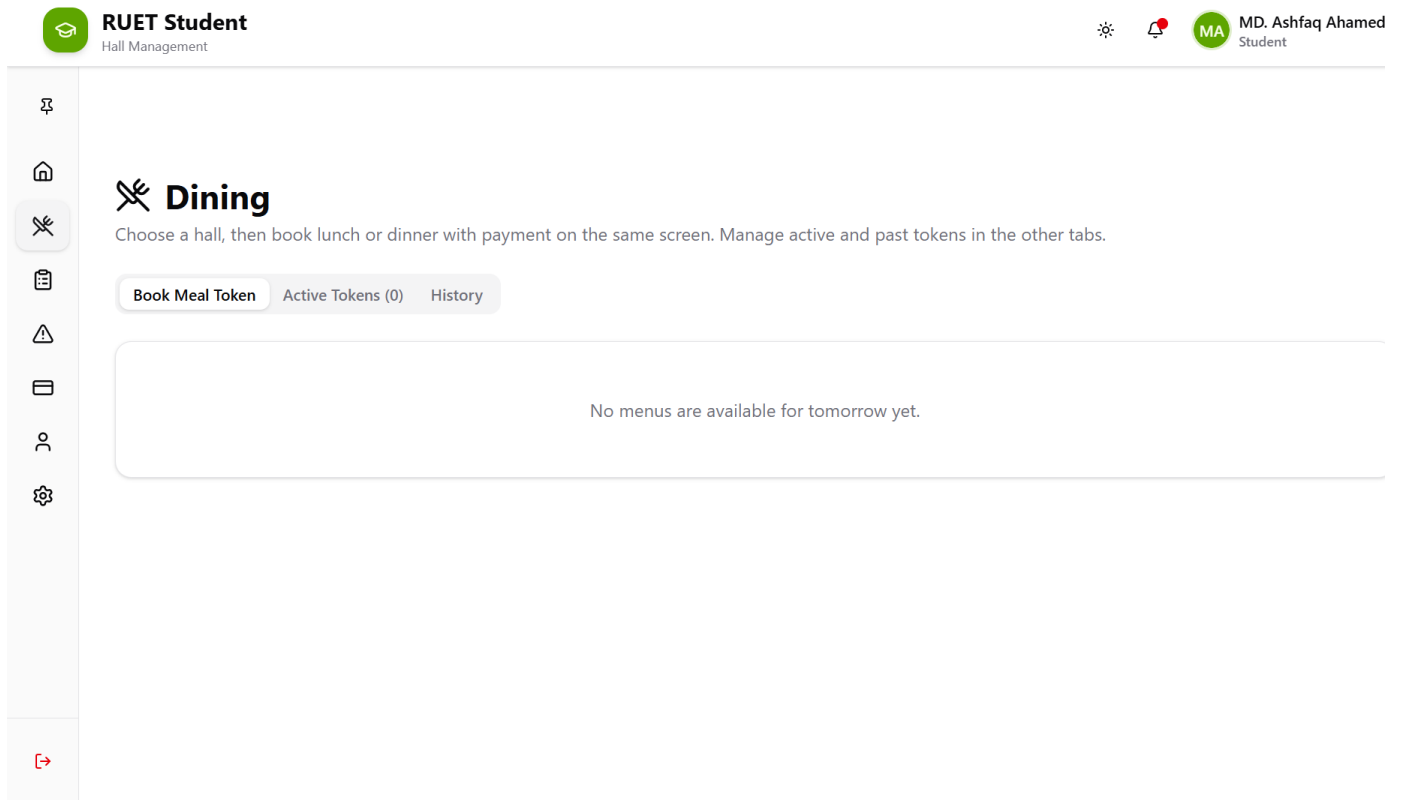


Figure 4 Meal Token Management Interface

Damage Reporting Interface

Students can submit reports regarding damaged hall facilities and track repair progress.

Report Damage
Submit a complaint using only the location description and asset details.

New Complaint

Location Description
Example: Zia Hall, Room 203, near the south window

Asset Details
Describe the damaged asset and what is wrong with it

Damage Proof Image
Choose File No file chosen
Upload a clear photo showing the damage.

Submit Complaint

RUET Student
Hall Management

Report Damage
Submit a complaint using only the location description and asset details.

New Complaint

Location Description
Example: Zia Hall, Room 203, near the south window

Asset Details
Describe the damaged asset and what is wrong with it
Please fill out this field.

Damage Proof Image
Choose File No file chosen
Upload a clear photo showing the damage.

Submit Complaint

Figure 5 Damage Reporting Interface

Notification Interface

The notification module displays important announcements and updates to students and administrators.

Figure 4.8: Notification Interface

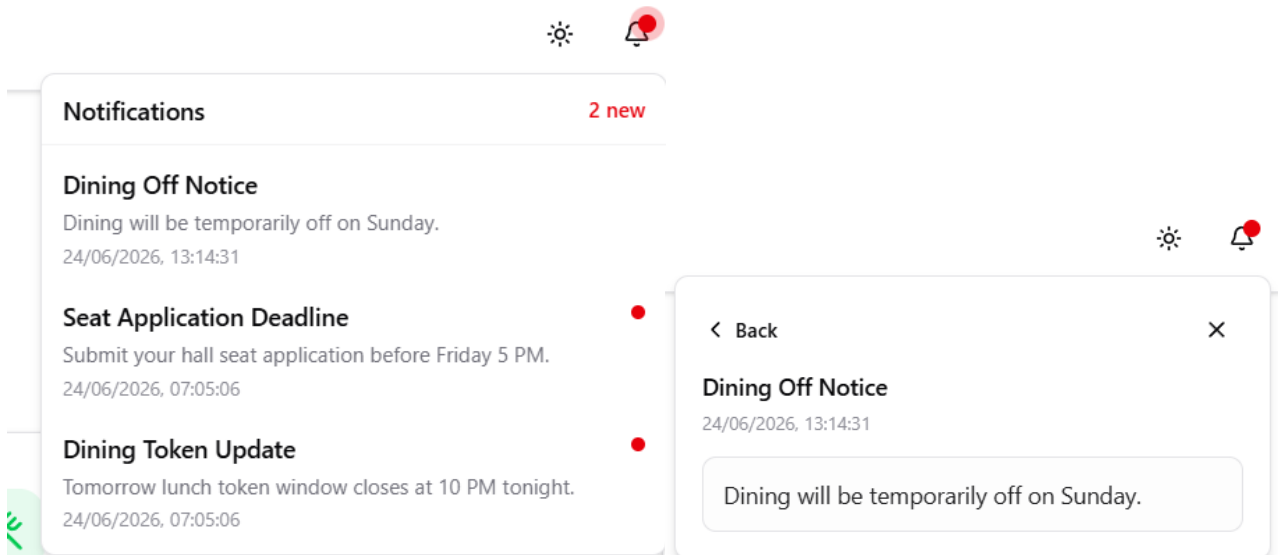


Figure 6 : Notification Interface

Implementation

5.1 Development Environment

- VS code

5.2 Technologies Used (Modern Tools)

- Nextjs, Expressjs, Postgresql

To build a robust and efficient system, the following modern tools were selected:

- **Next.js:** Chosen as the frontend framework for its high performance and developer-friendly features. Next.js facilitates fast, responsive user interfaces and supports server-side rendering, which optimizes page load times—essential for a seamless student experience.
- **Express.js:** Selected as the backend framework due to its lightweight and flexible nature. It simplifies the development of RESTful APIs, enabling efficient communication between the client and the server. Its extensive ecosystem and middleware support significantly accelerate the backend development process.
- **PostgreSQL:** Utilized as the relational database management system. It was selected for its exceptional reliability, strong data integrity, and excellent support for complex, structured data relationships, such as those between students, halls, rooms, and payment records. PostgreSQL ensures that financial and administrative data remain consistent and secure.

These modern tools significantly improved development efficiency by offering pre-built solutions for common web development challenges, modular architecture, and strong community support, allowing the development team to focus on core logic and feature implementation.

5.3 Module-wise Implementation

- **Admission Module:** Manages the online seat application process, allowing students to submit requests and administrators to review and approve them based on hall availability.
- **Auth Module:** Handles secure user registration and login, utilizing role-based access control to ensure authorized access for students, administrators, and the provost.

- **Dining Module:** Facilitates the creation of daily meal menus and allows students to book lunch and dinner tokens, providing real-time tracking of meal consumption and sales.
- **Finance Module:** Tracks hall fee payments, monitors student dues, and manages daily hall expense records, ensuring accurate financial calculations and transparency.
- **Inventory Module:** Enables the management of room occupancy and facility maintenance, allowing students to submit damage reports and administrators to track their repair status.
- **Notification Module:** Keeps users informed with system-wide announcements, seat approval updates, and payment confirmations to improve communication between authorities and students.
- **Payment Module:** Processes secure online transactions for hall fees and meal tokens, maintaining a digital record of all payments for verification and tracking.
- **Profile Module:** Stores and manages individual student information, academic details, and current hall assignment status, providing a personalized dashboard experience for each user.

5.4 Code Structure & Key Snippets

The following five snippets define how the backend handles requests: routing, authentication, sessions, validation, and the controller pattern.

1. Auth Middleware

This serves as the gatekeeper for every protected route.

```
// backend/src/middlewares/auth.middleware.ts
export const authenticateToken = async (
  req: Request,
  res: Response,
  next: NextFunction
) => {
  const sessionId = readSessionId(req);
  if (!sessionId) {
    throw new ApiError(401, "Session is required");
  }
  const liveSession = await getSession(sessionId);
  if (!liveSession) {
    clearSessionCookie(res);
    throw new ApiError(401, "Session is invalid or has expired");
  }
}
```

```
}  
};
```

Functionality: Reads the session from a cookie or *Authorization: Bearer* header, loads it from Redis, hydrates the user from PostgreSQL, and attaches *req.user*. Invalid or deactivated accounts are rejected, and sessions are revoked. This runs before almost every business endpoint.

2. Session Store

This handles the logic for user login.

```
// backend/src/lib/sessionStore.ts  
export async function createSession(  
  req: Request,  
  payload: SessionUserPayload  
) : Promise<LiveSession> {  
  const redis = await requireRedis();  
  const sessionId = randomUUID();  
  const record: LiveSession = {  
    sessionId,  
    ...payload,  
    ip: String(req.ip ?? ""),  
    userAgent: req.headers["user-agent"] ?? "",  
    createdAt: new Date().toISOString(),  
  };  
  await redis.set(sessionKey(sessionId), JSON.stringify(record), {  
    EX: sessionTtlSec,  
  });  
  await redis.sAdd(userSessionsKey(payload.userId), sessionId);  
  return record;  
}
```

Functionality: On login, creates a UUID session in Redis (max 2 devices per user), sets an *httpOnly* cookie, and uses a sliding TTL on each request. Auth is server-side session state, not a signed JWT.

3. App Bootstrap

The central API map.

```
// backend/src/app.ts  
app.use("/api/auth", authRouter);  
app.use("/api/profile", profileRouter);  
app.use("/api/dining", diningRouter);  
app.use("/api/admission", admissionRouter);  
app.use("/api/inventory", inventoryRouter);
```

```
app.use("/api/finance", financeRouter);
app.use("/api/payments", paymentRouter);
app.use("/api/notifications", notificationsRouter);
```

Functionality: Mounts all feature modules under */api/**. Each module is a self-contained folder with *routes* → *controller* → *Drizzle*. This is the single place to view the backend's exposed endpoints.

4. Request Validation

Validation using Zod on every route.

```
// backend/src/middlewares/validateRequest.middleware.ts
export const validateRequest =
  (schemas: ValidationSchemas) =>
  async (req: Request, _res: Response, next: NextFunction) => {
    const result = await schema.safeParseAsync(req[key]);
    if (!result.success) {
      return next(formatZodError(result.error));
    }
    Object.defineProperty(req, key, {
      value: result.data,
      writable: true,
      configurable: true,
    });
  };
};
```

Functionality: Validates *body*, *params*, *query*, etc., against Zod schemas from **.validators.ts* before the controller runs. This ensures controllers can trust *req.body*, as invalid input never reaches the business logic.

5. Controller Pattern

The standard controller recipe used across all modules.

```
// backend/src/modules/auth/auth.controller.ts
export const studentLogin = async (req: Request, res: Response) => {
  const { email, password } = req.body;
  const [user] = await db
    .select()
    .from(uniStudents)
    .where(eq(uniStudents.email, email))
    .limit(1);
  if (!user) {
    throw new ApiError(401, "No user found with this email");
  }
}
```

```
// ... password + status checks
await enforceSessionLimitOrThrow(user.id);
await createSessionAndSetCookie({ req, res, payload: sessionPayload });
return res.status(200).json(
  new ApiResponse(200, { student_data: toStudentLoginData(user) },
    "User logged in successfully")
);
};
```

Functionality: The standard recipe: 1. Query with **Drizzle** (no raw SQL). 2. Enforce rules via *throw new ApiError(status, message)* (no *try/catch*). 3. Handle side effects (sessions, emails, payments). 4. Return *new ApiResponse(status, data, message)*. Express 5 catches the error, and the global handler formats the JSON response.

How they connect:

Request → Route → authenticateToken → authorizeRoles → validateRequest → Controller → ApiResponse

↓

throw ApiError

Chapter 6

Testing & Validation

6.1 Testing Strategy

To ensure the reliability and correctness of the proposed Hall Management System, several testing approaches were conducted during the development phase. Both individual modules and the complete system were tested to identify and eliminate errors before deployment.

The testing process focused on validating user authentication, hall seat application, meal token management, payment tracking, and complaint submission functionalities.

The following testing methods were applied:

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing

6.2 System Testing

System testing was performed to verify that all modules function correctly when integrated into a complete system.

Table 2 Sample Test Cases

Test Case	Input	Expected Result	Status
Student Login	Valid Email & Password	Login Successful	Passed
Student Login	Invalid Password	Error Message Displayed	Passed
Hall Application	Valid Information	Application Submitted	Passed
Meal Token Purchase	Valid Request	Token Generated	Passed
Complaint Submission	Valid Complaint	Complaint Recorded	Passed
Admin Approval	Approve Request	Status Updated	Passed

The results indicate that the developed system successfully performs the intended operations without significant errors.

6.3 User Acceptance Testing

User Acceptance Testing (UAT) was conducted with a group of students and hall management personnel.

The users were asked to perform common tasks such as:

- Registering an account

- Applying for a hall seat
- Purchasing meal tokens
- Submitting complaints
- Reviewing application status

Feedback from the participants indicated that the system is easy to use and significantly reduces the effort required for managing hall-related activities. Most users expressed satisfaction with the centralized management features and user-friendly interface.

Chapter 7

Project Management & Teamwork

7.1 Project Planning

The project was divided into several phases to ensure systematic development.

The major activities included:

- Requirement Analysis
- System Design
- Database Design
- Frontend Development
- Backend Development
- Testing and Debugging
- Documentation

This structured planning helped complete the project within the allocated timeframe.

7.2 Individual & Team Work

The project was completed through collaborative efforts among team members. Responsibilities were distributed based on individual skills and interests.

Major Contributions

- Requirement Analysis and Planning
- Database Design
- Frontend Development
- Backend Development
- Testing and Documentation

Regular discussions and meetings were conducted to monitor progress and resolve technical challenges.

Version control and collaborative development practices were followed throughout the project.

Challenges Faced

Some challenges encountered during development included:

- Database relationship management
- User authentication implementation
- Integration of different modules
- Debugging backend functionalities

These challenges were addressed through research, testing, and team collaboration.

7.3 Timeline

Table 3 Project Timeline

Activity	Duration
Requirement Analysis	Week 1
System Design	Week 2
Database Design	Week 3-4
Frontend Development	Week 5-7
Backend Development	Week 8-10
Testing & Debugging	Week 11
Documentation	Week 12

7.4 Budget & Cost Analysis

The project was developed primarily using open-source technologies, minimizing development expenses.

Table 4 Estimated Cost Analysis

Item	Estimated Cost (BDT)
Development Tools	0
Database Software	0
Hosting (Optional)	2,000
Domain (Optional)	1,000
Miscellaneous Expenses	1,000
Total Estimated Cost	4,000

Since most technologies used are freely available, the implementation cost remains relatively low.

7.5 Risk Management

Several risks were identified during project development.

Table 5 Risk Assessment

Risk	Impact	Mitigation Strategy
Data Loss	High	Regular Database Backup
Unauthorized Access	High	Authentication & Security Controls
Server Failure	Medium	Backup Hosting Plan
Software Bugs	Medium	Comprehensive Testing

Proper preventive measures were implemented to minimize these risks.

Chapter 8

Results & Discussion

8.1 System Output

The developed Hall Management System successfully provides the required functionalities for students and hall authorities.

Major outputs of the system include:

- Student Registration
- Secure Login
- Hall Seat Application
- Meal Token Generation
- Payment Information Management
- Complaint Submission
- Administrative Dashboard

The generated outputs demonstrate that the system effectively automates hall management activities.

Figures

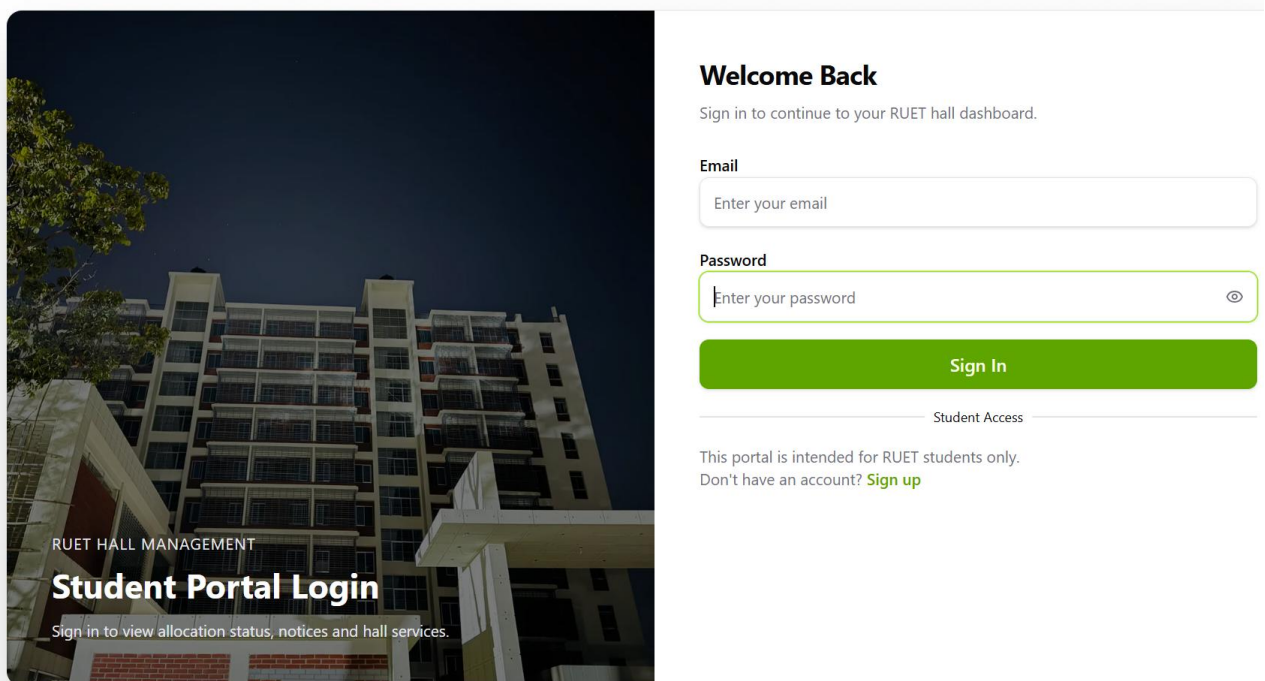


Figure 6 Login Page

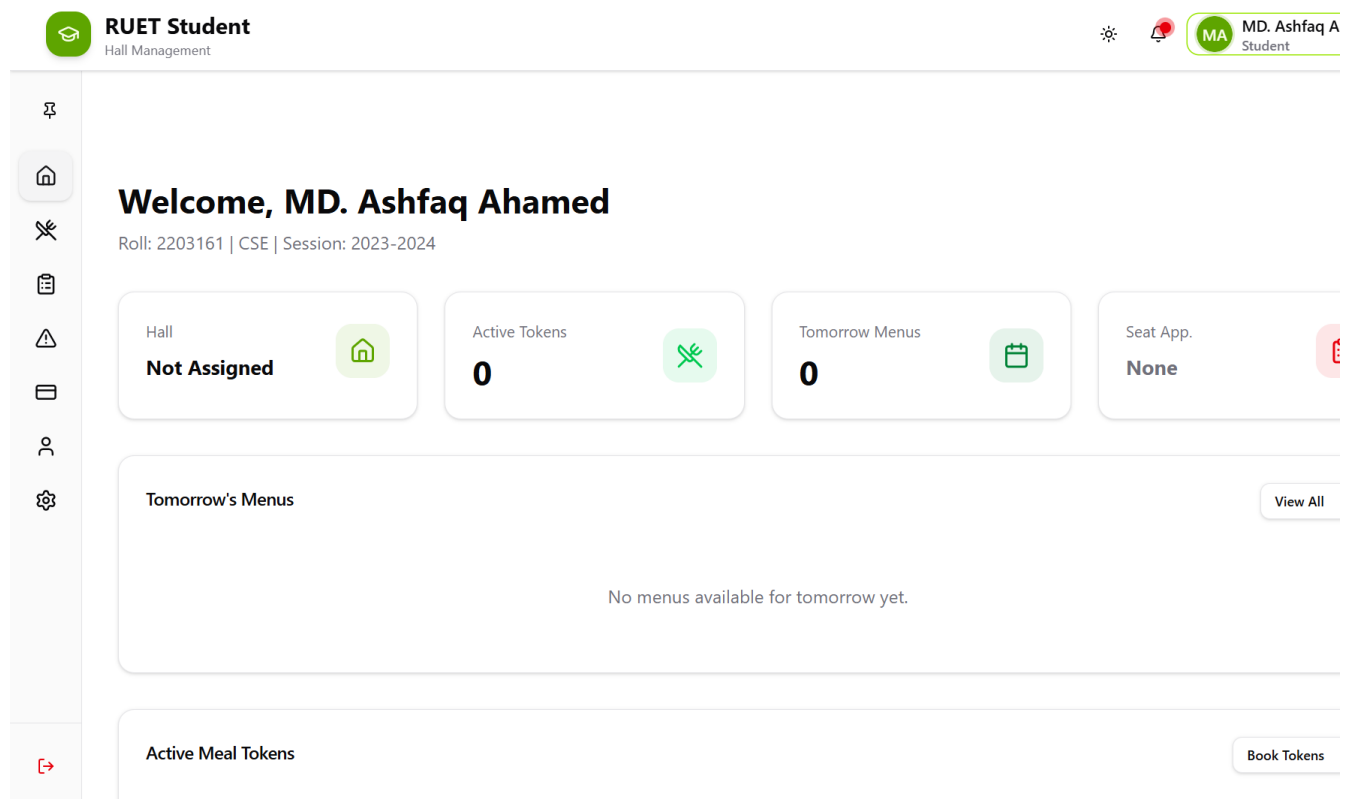
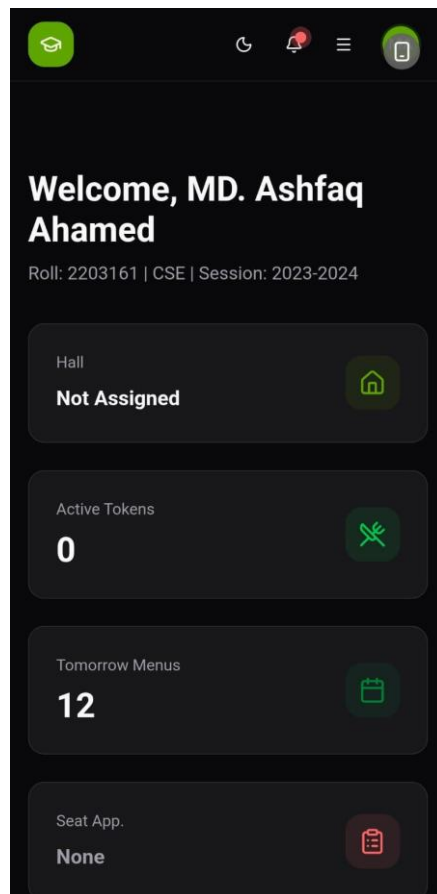


Figure 7 Student Dashboard

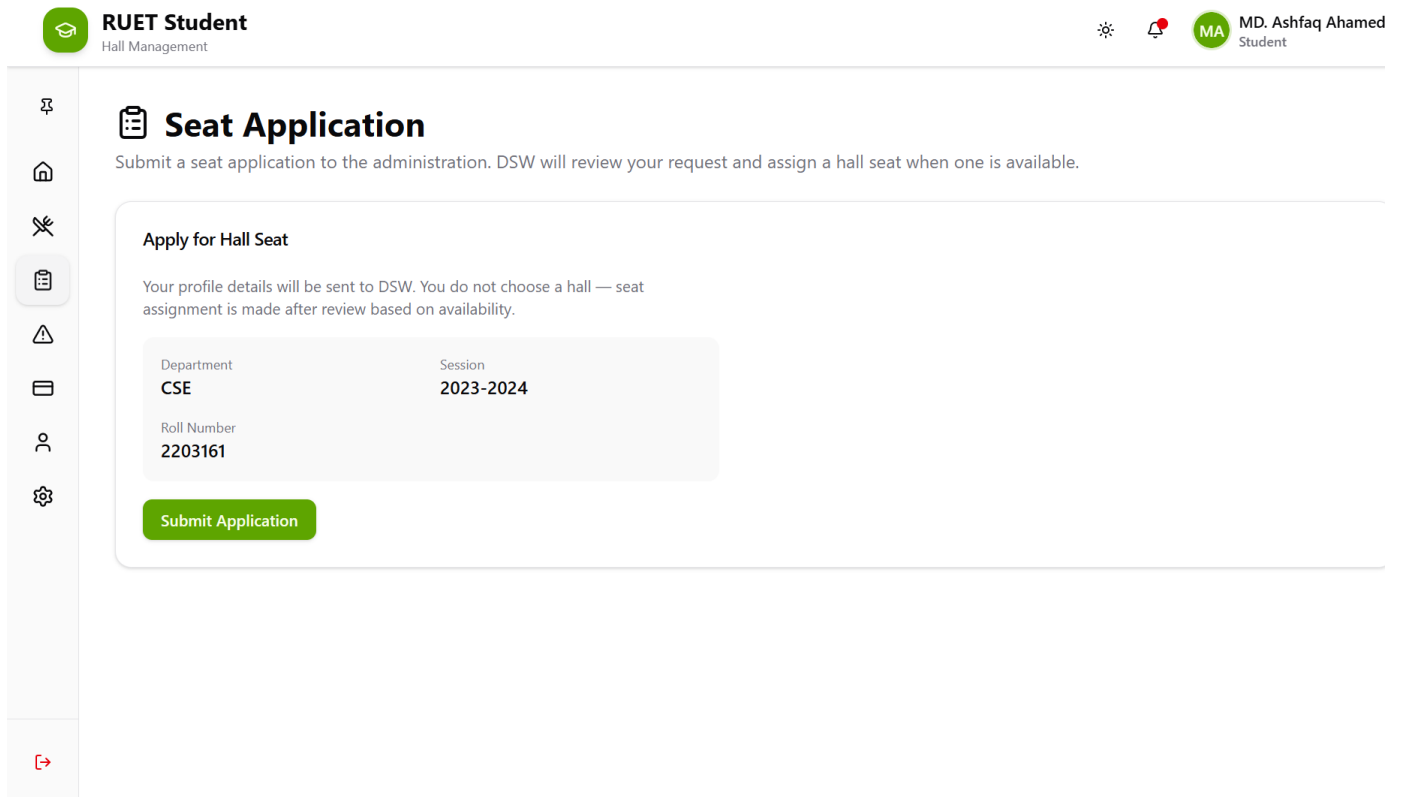
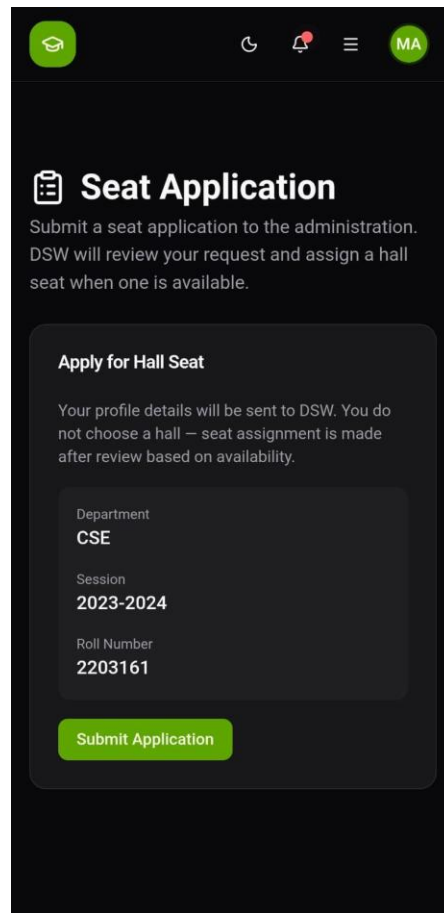


Figure 8 Hall Application Page

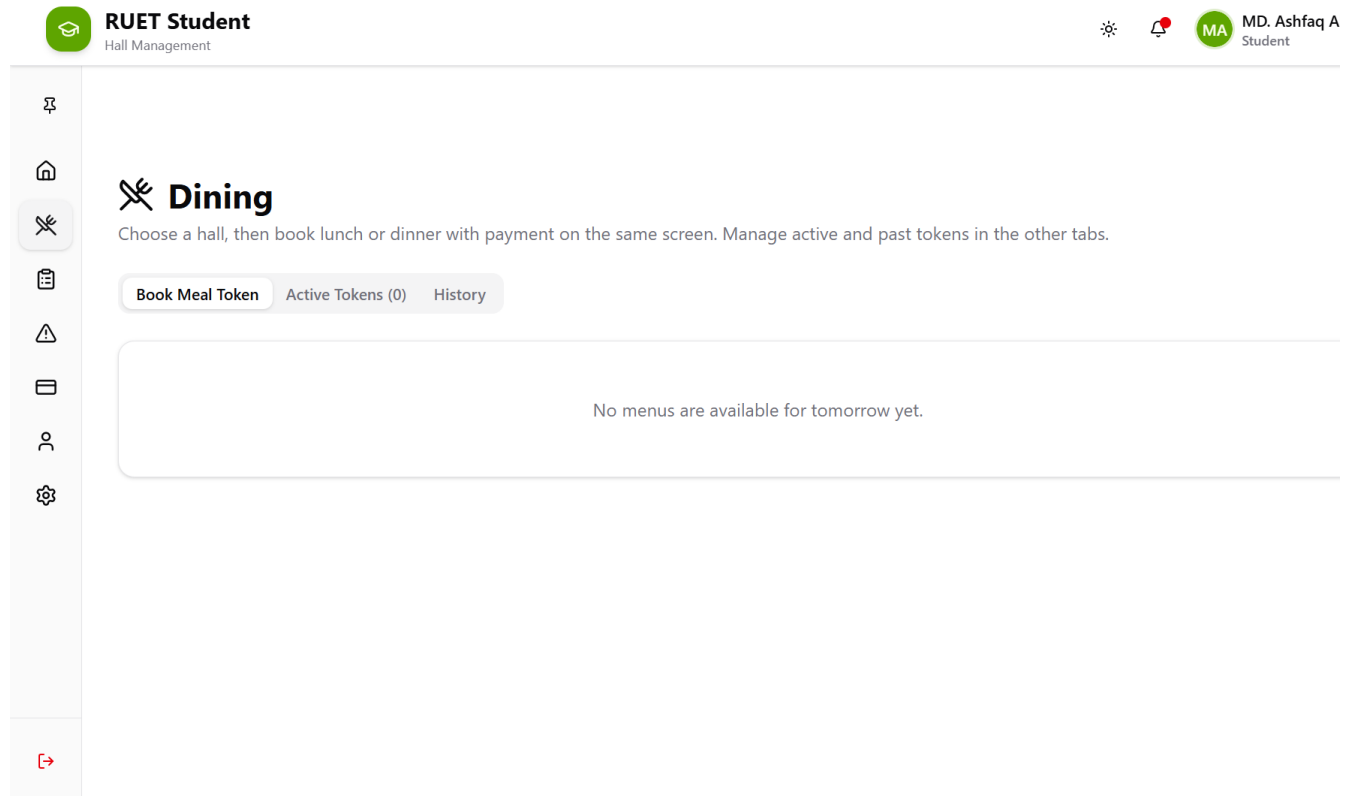
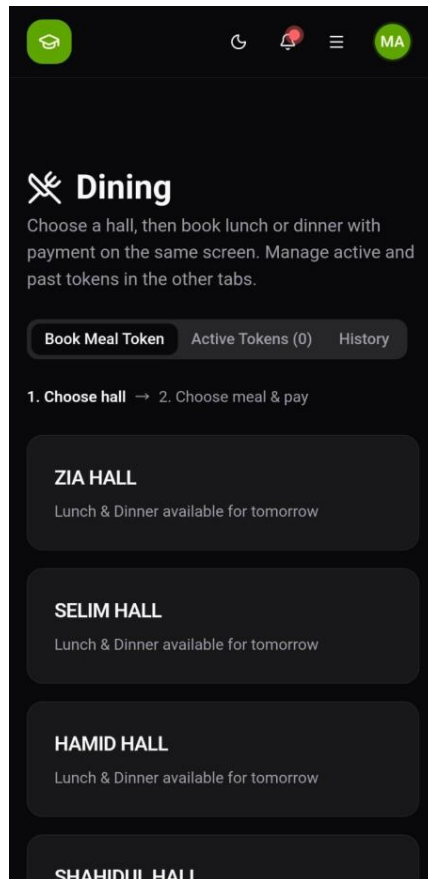


Figure 9 Token Book



Report Damage

Submit a complaint using only the location description and asset details.

New Complaint

Location Description

Example: Zia Hall, Room 203, near the south window

Asset Details

Describe the damaged asset and what is wrong with it

Damage Proof Image

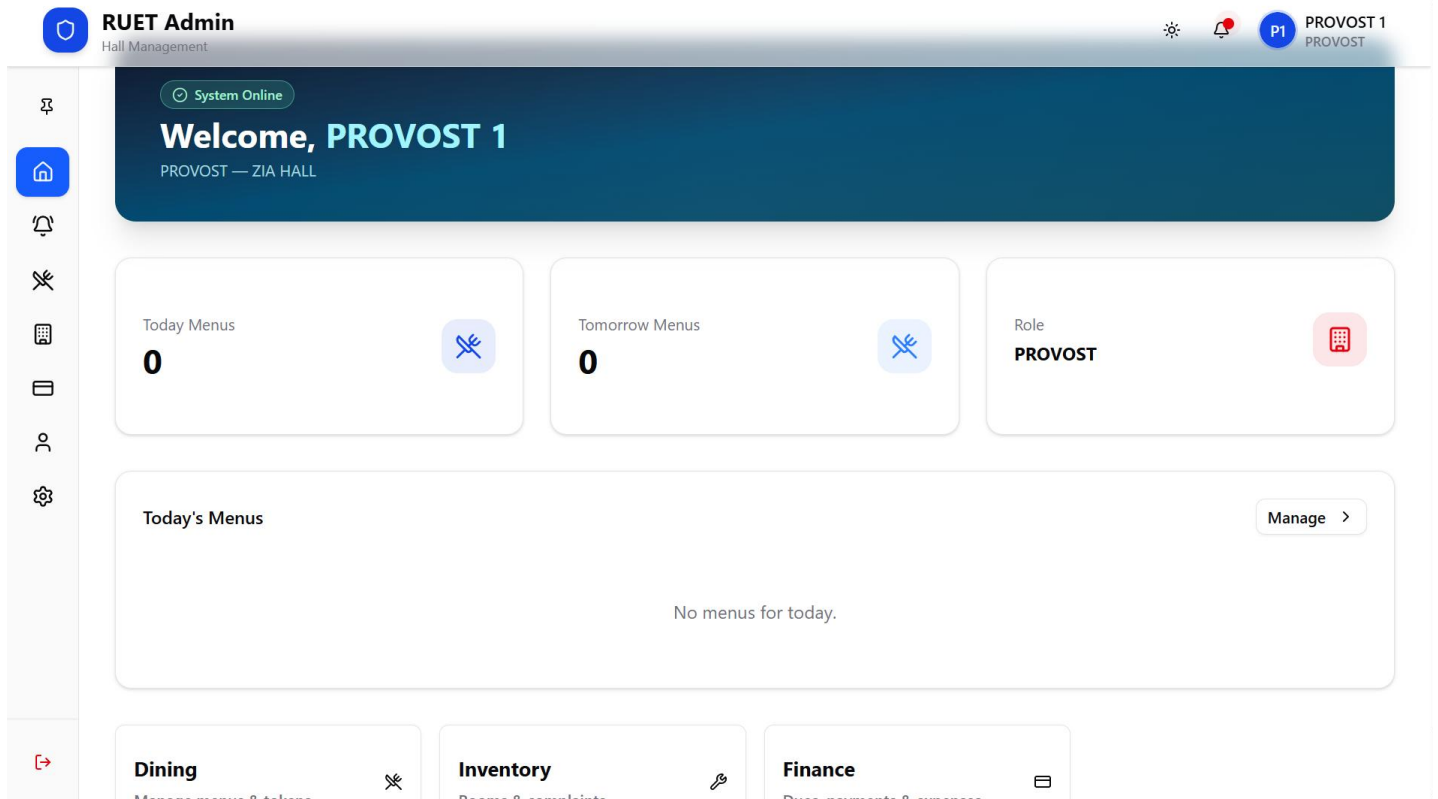
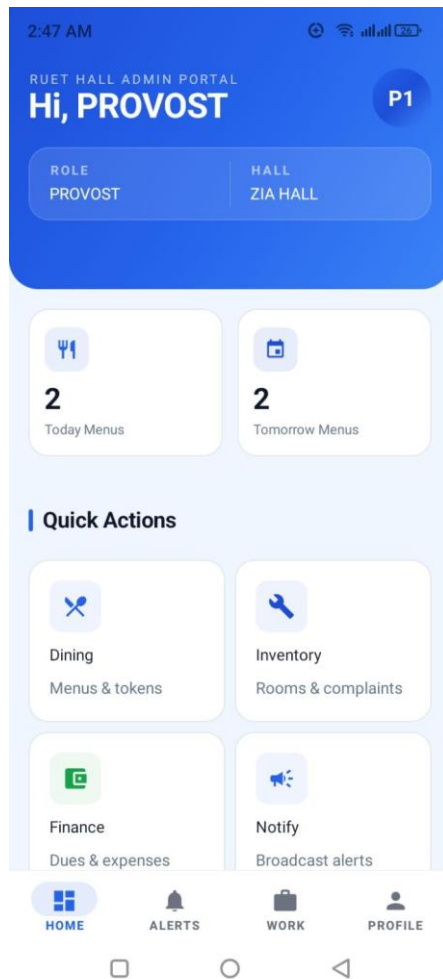
Choose File No file chosen

Upload a clear photo showing the damage.

Submit Complaint

[illegible]

Figure 10 Complaint Management Page



8.2 Performance Evaluation

The system was evaluated based on responsiveness, usability, and functionality.

Table 6 Performance Evaluation

Criteria	Result
Login Response Time	Fast
Database Query Performance	Efficient
User Interface Usability	Good
System Reliability	High
Functional Accuracy	High

The results indicate satisfactory system performance under normal operating conditions.

8.3 Comparison with Existing Systems

Table 7 Comparison with Existing Systems

Feature	Existing Manual System	Proposed System
Online Seat Application	No	Yes
Meal Token Management	No	Yes
Complaint Management	No	Yes
Automated Records	No	Yes
Report Generation	Difficult	Easy
Centralized Data Storage	No	Yes

The proposed system offers significant improvements over traditional hall management approaches.

Chapter 9

Conclusion & Future Work

9.1 Conclusion

The primary objective of this project was to develop a complete web-based Hall Management System for Rajshahi University of Engineering & Technology (RUET). The developed system successfully automates several hall management activities that are traditionally performed manually.

Through the system, students can create accounts, apply for hall seats, purchase meal tokens, make online payments, and submit complaints regarding hall facilities. On the other hand, hall administrators can manage student applications, monitor payments, track meal token sales, and generate reports in PDF format with a single click.

The system reduces paperwork, minimizes administrative workload, improves transparency, and provides a more convenient experience for both students and hall authorities. The successful implementation of this project demonstrates the effectiveness of digital solutions in modernizing residential hall management.

Overall, the developed system fulfills the project requirements and provides a practical solution for efficient hall administration at RUET.

9.2 Limitations

Although the system successfully implements the core functionalities, a few limitations still exist.

- The system is currently designed specifically for RUET residential halls.
- High server load may affect performance if a very large number of users access the system simultaneously.
- The current reporting system focuses mainly on hall management activities and may require additional customization for future administrative requirements.
- Multi-language support has not been implemented in the current version.

Despite these limitations, the system remains fully functional and capable of supporting day-to-day hall management operations.

9.3 Future Enhancements

The developed system provides a strong foundation for future improvements. Several additional features can be incorporated to make the system more powerful and scalable.

Future enhancements may include:

- Mobile application development for Android and iOS platforms.
- QR Code or RFID-based meal token verification.
- Push notifications for seat approval, payment confirmation, and announcements.
- AI-based analytics for meal demand prediction and hall resource management.
- Real-time occupancy monitoring of hall rooms.
- Integration with the university's central student information system.
- Advanced data visualization dashboards for hall authorities.
- Automated maintenance tracking and work-order management.
- Multi-hall centralized administration under a single platform.
- Cloud deployment for improved scalability and reliability.

These enhancements would further improve efficiency, user experience, and the overall effectiveness of the Hall Management System.

REFERENCES

- Next.js Documentation: <https://nextjs.org/docs>
- Express.js Documentation: <https://expressjs.com/>
- PostgreSQL Documentation: <https://www.postgresql.org/>
- Drizzle ORM Documentation: <https://orm.drizzle.team/docs/overview>
- Redis Documentation: <https://redis.io/docs/latest/get-started/>
- Cloudinary Documentation: https://cloudinary.com/documentation/programmable_media_guides
- Brevo API Documentation: <https://developers.brevo.com/>
- Project Repository: <https://github.com/aniruddha81/ruet-hall-mgmt>

APPENDICES

Appendix A: Project Source Code Structure

The project follows a modular architecture:

- /backend/src/modules/: Contains feature-specific modules (auth, dining, admission, etc.) with routes, controllers, and validators.
- /backend/src/middlewares/: Contains global middleware for auth and request validation.
- /backend/src/lib/: Contains shared utilities and database/session store configurations.
- /frontend/: Contains the Next.js application with UI components and dashboard layouts.

Appendix B: Database Schema Definition

The system uses a PostgreSQL database with the following core tables:

- uni_students: Stores student profiles and academic info.
- hall_admins: Manages admin/provost credentials and roles.
- meal_tokens: Records meal bookings and history.
- damage_reports: Stores complaints and repair status.
- payments: Logs hall fee and meal payment transactions.

Appendix C: User Manual and Deployment Guide

- Environment Setup: Ensure Node.js (v20+), PostgreSQL (v16+), and Redis are installed.
- Configuration: Copy .env.example to .env and configure DATABASE_URL, REDIS_URL, and Cloudinary/Brevo API keys.
- Installation: Run npm install in both /backend and /frontend directories.
- Database: Run npx drizzle-kit push to synchronize the schema.
- Start: Use npm run dev to start the development servers.